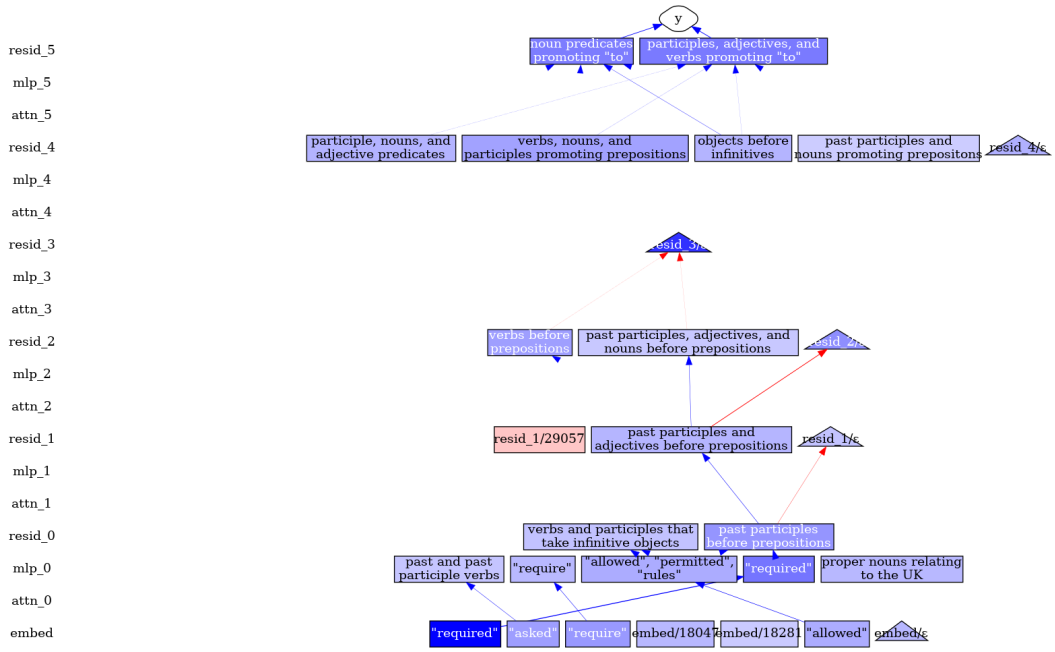

Figure 18: The full annotated feature circuit for predicting "to" as an infinitival object. The model first detects the presence of verbs that often take infinitival objects. Then, it uses one mechanism to detect present-tense verbs, participles, or predicate adjectives which take infinitival objects, and another mechanism to detect direct objects that can directly precede infinitival object complements. Finally, these two mechanisms both influence the output in layer 5 without fully intersecting.

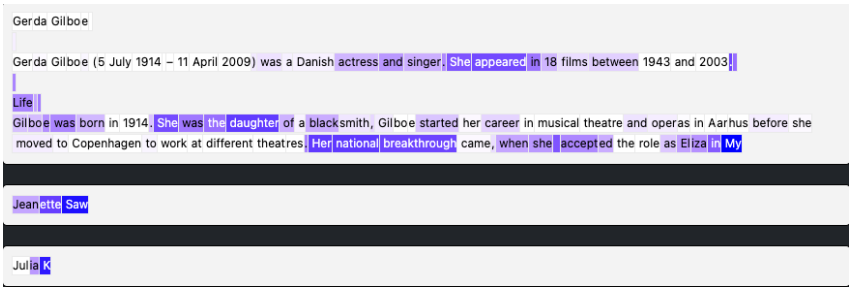

Figure 19: An example sparse feature from the Bias in Bios task (`attn_3/22029`). This feature detects female-related words in biographies of women. It also promotes words like "husband" and "née". This feature probably contributes to preferences for the spurious correlate of gender; we therefore ablate it.

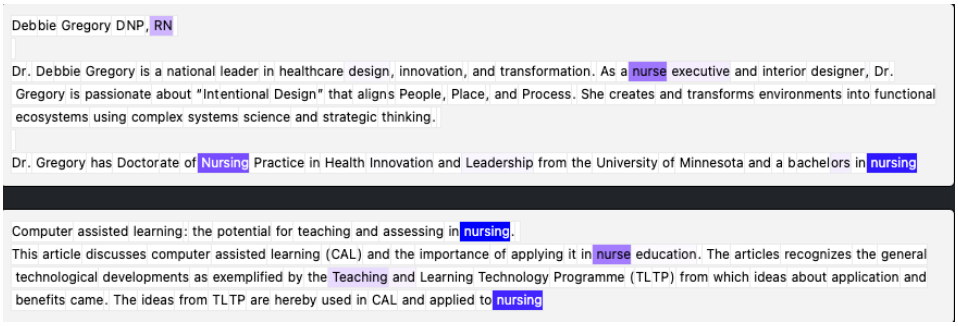

Figure 20: An example sparse feature from the Bias in Bios task (`resid_2/31098`). This feature activates on words related to nursing, including "RN" and "nurse". This probably relates to the target task of profession prediction. We therefore keep it.

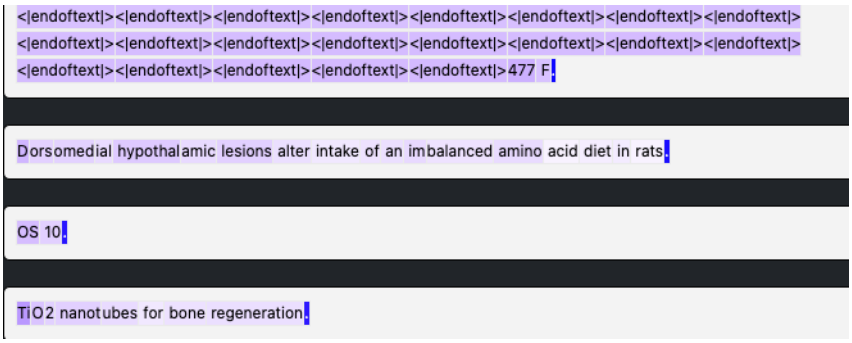

Figure 21: An example neuron from the Bias in Bios task. This appears to activate on beginnings and ends of sentences, but also more strongly on any token in a sentence that contains capital letters or numbers. We cannot deduce whether this would contribute more to gender or profession names.

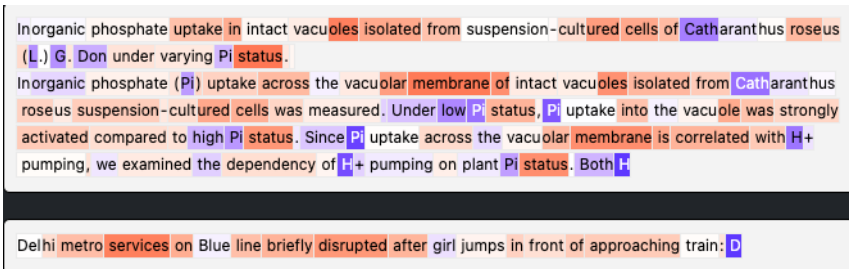

Figure 22: An example neuron from the Bias in Bios task. This activates positively on tokens starting with capital letters, but negatively on many other tokens (whose unifying theme we cannot deduce).

E.2 Implementation for Concept Bottleneck Probing

Our implementation for Concept Bottleneck Proving (CBP) is adapted from (Yan et al., 2023). It works as follows:

1. First, we collect a number of keywords related to the intended prediction task. We use $N = 20$ keywords: nurse, healthcare, hospital, patient, medical, clinic, triage, medication, emergency, surgery, professor, academia, research, university, tenure, faculty, dissertation, sabbatical, publication, and grant.
2. We obtain *concept vectors* $\mathbf{c}_1, \ldots, \mathbf{c}_N$ for each keyword by extracting Pythia-70M's penultimate layer representation over the final token of each keyword, and then subtracting off the mean concept vector. (Without this normalization, we found that concept vectors have very high pairwise cosine similarities.)
3. Given an input with representation $\mathbf{x}$ (obtained via the mean-pooling procedure in App. E.1), we obtain a concept bottleneck representation $\mathbf{z} \in \mathbb{R}^N$ by taking the cosine similarity with each $\mathbf{c}_i$.
4. Finally, we train a linear probe with logistic regression on the concept bottleneck representations $\mathbf{z}$, as in App. E.1.

We decided to normalize concept vectors but not input representations because it resulted in stronger performance. We also experimented with computing cosine similarities before mean pooling.

F Human Interpretability Ratings for Sparse Features

Given our trained Pythia-70M sparse autoencoders, we asked human crowdworkers to rate the interpretability of random features, random neurons, features from our feature circuits, and neurons from our neuron circuits on a 0–100 scale (Table 7). Crowdworkers rate sparse features as significantly more interpretable than neurons, with features that participate in our circuits also being more interpretable than randomly sampled features.

Activation type	Interpretability
Dense (random)	32.6
Dense (agreement)	30.2
Dense (BiB)	36.0
Sparse (random)	52.8
Sparse (agreement)	62.3
Sparse (BiB)	81.5

Table 7: Human interpretability ratings for dense (neuron) vs. sparse (autoencoder) features. We present mean interpretability scores across features on a 0–100 scale. We show scores for features that were either uniformly sampled (random), the top 30 by $\hat{\text{IE}}$ from the subject-verb agreement across RC task (agreement; §3.3), or the top 30 by $\hat{\text{IE}}$ for the Bias in Bios task (BiB; §4).

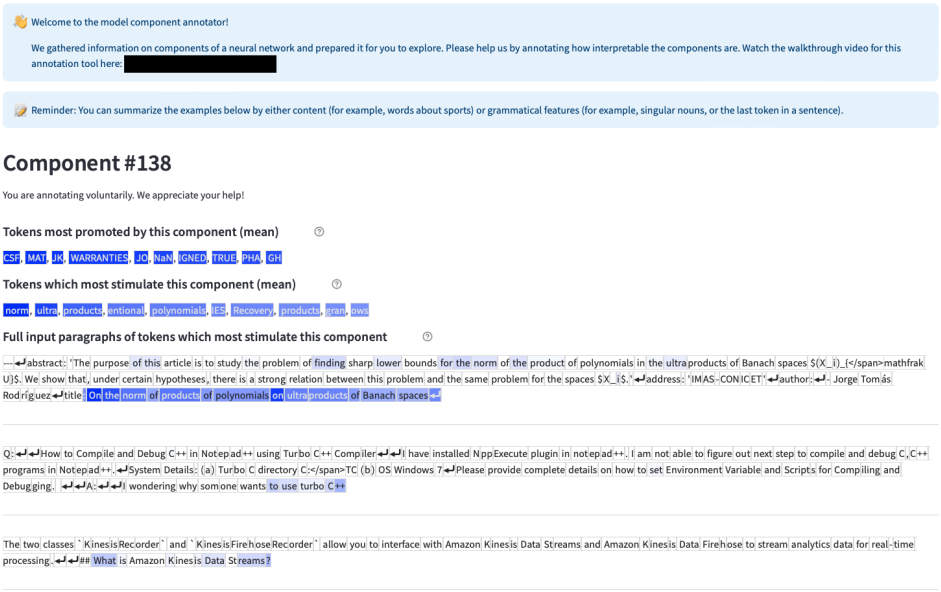

Figure 23: The human annotation interface used to obtain the interpretability ratings in Table 7. Here, we show the instructions, top-activating tokens, the token probabilities that were most affected when ablating the feature, and example contexts with feature activation values.